```
import pandas as pd
import numpy as np
from google.colab import files
uploaded = files.upload()
```

```
Choose files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving yeast.csv to yeast.csv
```

```
df = pd.read_csv("yeast.csv")

print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())

print("\nFirst 5 rows:")
display(df.head())

print("\nMissing values:")
print(df.isnull().sum())
```

```
Dataset shape: (1484, 9)

Column names:
['mcg', 'gvh', 'alm', 'mit', 'erl', 'pox', 'vac', 'nuc', 'name']

First 5 rows:
```

	mcg	gvh	alm	mit	erl	pox	vac	nuc	name
0	0.58	0.61	0.47	0.13	0.5	0.0	0.48	0.22	MIT
1	0.43	0.67	0.48	0.27	0.5	0.0	0.53	0.22	MIT
2	0.64	0.62	0.49	0.15	0.5	0.0	0.53	0.22	MIT
3	0.58	0.44	0.57	0.13	0.5	0.0	0.54	0.22	NUC
4	0.42	0.44	0.48	0.54	0.5	0.0	0.48	0.22	MIT

```
Missing values:
mcg     0
gvh     0
alm     0
mit     0
erl     0
pox     0
vac     0
nuc     0
name    0
dtype: int64
```

```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# Select numeric columns
numeric_df = df.select_dtypes(include=np.number)

# Remove columns with no usable values
numeric_df = numeric_df.dropna(axis=1, how="all")

# Fill missing values
imputer = SimpleImputer(strategy="median")
X = pd.DataFrame(
    imputer.fit_transform(numeric_df),
    columns=numeric_df.columns,
    index=df.index
)

# Standardize features for KNN
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Numeric features used:")
print(X.columns.tolist())
print("Prepared data shape:", X.shape)
```

```
Numeric features used:
['mcg', 'gvh', 'alm', 'mit', 'erl', 'pox', 'vac', 'nuc']
Prepared data shape: (1484, 8)
```

IQR

```
Q1 = X.quantile(0.25)
Q3 = X.quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

iqr_flags = (X.lt(lower_bound) | X.gt(upper_bound))

df["IQR_Outlier"] = iqr_flags.any(axis=1)
```

```
print("IQR outliers:", df["IQR_Outlier"].sum())
print("\nOutlier records:")
display(df[df["IQR_Outlier"]].head(10))
```

```
IQR outliers: 408

Outlier records:
```

	mcg	gvh	alm	mit	erl	pox	vac	nuc	name	IQR_Outlier
5	0.51	0.40	0.56	0.17	0.5	0.5	0.49	0.22	CYT	True
6	0.50	0.54	0.48	0.65	0.5	0.0	0.53	0.22	MIT	True
29	0.80	0.88	0.36	0.39	0.5	0.0	0.56	0.33	ME1	True
31	0.51	0.51	0.43	0.87	0.5	0.0	0.52	0.22	MIT	True
32	0.38	0.61	0.61	0.12	0.5	0.0	0.53	0.47	CYT	True
40	0.86	0.68	0.39	0.24	0.5	0.0	0.54	0.22	ME2	True
41	0.86	0.74	0.51	0.40	0.5	0.0	0.55	0.22	ME2	True
46	0.74	0.75	0.58	0.47	0.5	0.0	0.40	0.35	ME2	True
51	0.41	0.53	0.45	0.14	0.5	0.0	0.52	0.66	ME3	True
59	0.45	0.58	0.48	0.67	0.5	0.0	0.52	0.22	MIT	True

Z-Score

```
from scipy.stats import zscore

z_scores = pd.DataFrame(
    zscore(X, axis=0, ddof=0),
    columns=X.columns,
    index=X.index
)

z_flags = z_scores.abs() > 3

df["ZScore_Outlier"] = z_flags.any(axis=1)

print("Z-score outliers:", df["ZScore_Outlier"].sum())
print("\nOutlier records:")
display(df[df["ZScore_Outlier"]].head(10))
```

```
Z-score outliers: 126

Outlier records:
```

	mcg	gvh	alm	mit	erl	pox	vac	nuc	name	IQR_Outlier	ZScore_Outlier
5	0.51	0.40	0.56	0.17	0.5	0.50	0.49	0.22	CYT	True	True
29	0.80	0.88	0.36	0.39	0.5	0.00	0.56	0.33	ME1	True	True
31	0.51	0.51	0.43	0.87	0.5	0.00	0.52	0.22	MIT	True	True
51	0.41	0.53	0.45	0.14	0.5	0.00	0.52	0.66	ME3	True	True
84	0.67	0.62	0.40	0.43	0.5	0.83	0.50	0.22	MIT	True	True
85	0.28	0.58	0.21	0.13	0.5	0.00	0.51	0.22	ME3	True	True
110	0.45	0.41	0.50	0.46	0.5	0.50	0.50	0.27	POX	True	True
114	0.45	0.51	0.50	0.16	0.5	0.00	0.52	1.00	NUC	True	True
118	0.35	0.52	0.48	0.22	0.5	0.00	0.26	0.22	MIT	True	True
148	0.26	0.36	0.47	0.34	0.5	0.00	0.53	0.72	NUC	True	True

KNN

```
from sklearn.neighbors import NearestNeighbors

k = 5

if len(X) <= k:
    raise ValueError("Dataset must contain more than 5 rows.")

knn = NearestNeighbors(n_neighbors=k + 1)
knn.fit(X_scaled)

distances, indices = knn.kneighbors(X_scaled)

# First neighbor is the record itself, so exclude it
knn_scores = distances[:, 1:].mean(axis=1)

# Flag records above the 95th percentile
threshold = np.percentile(knn_scores, 95)

df["KNN_Score"] = knn_scores
df["KNN_Outlier"] = knn_scores > threshold
```

```
print("KNN distance threshold:", threshold)
print("KNN outliers:", df["KNN_Outlier"].sum())

display(
    df[df["KNN_Outlier"]]
    .sort_values("KNN_Score", ascending=False)
    .head(10)
)
```

```
KNN distance threshold: 1.7236030565354379
KNN outliers: 75
```

	mcg	gvh	alm	mit	erl	pox	vac	nuc	name	IQR_Outlier	ZScore_Outlier	KNN_Score	KNN_Outlier
553	0.18	0.48	0.46	0.12	0.5	0.0	0.00	0.25	NUC	True	True	4.681443	True
989	0.35	0.21	1.00	0.80	0.5	0.0	0.13	0.01	CYT	True	True	4.421959	True
990	0.35	0.21	1.00	0.80	0.5	0.0	0.13	0.01	CYT	True	True	4.421959	True
988	0.43	0.22	0.79	0.79	0.5	0.0	0.22	0.00	CYT	True	True	3.765367	True
1125	0.66	0.43	0.57	0.60	0.5	0.0	0.19	0.33	NUC	True	True	3.568823	True
841	0.33	0.21	0.53	0.15	1.0	0.0	0.52	0.50	NUC	True	True	3.374919	True
229	0.37	0.53	0.60	0.19	0.5	0.5	0.42	0.22	POX	True	True	3.357940	True
470	0.89	0.81	0.36	0.71	0.5	0.0	0.55	0.63	ME1	True	True	3.307481	True
488	0.85	0.56	0.33	0.38	1.0	0.0	0.55	0.25	ERL	True	True	3.295072	True
110	0.45	0.41	0.50	0.46	0.5	0.5	0.50	0.27	POX	True	True	3.179848	True